MA35D1 TensorFlow Lite Developer's Guide

Application Note for NuMicro® MA35D1 Family

Document Information

Abstract	This document provides a guide on how to develop TensorFlow Lite TinyML applications on MA35D1 platform.
Apply to	NuMicro® MA35D1 series.
Author	江天文 (NTSH)

For additional information or questions, please contact: Nuvoton Technology Corporation.

www.nuvoton.com

Table of Contents

1 Overview

TensorFlow is an end-to-end open-source platform for machine learning. It has a comprehensive, flexible ecosystem of tools, libraries and community resources that lets researchers push the state-of-the-art in ML and developers easily build and deploy ML-powered applications.

TensorFlow was originally developed by researchers and engineers working within the Machine Intelligence team at Google Brain to conduct research in machine learning and neural networks. However, the framework is versatile enough to be used in other areas as well.

TensorFlow provides stable Python and C++ APIs, as well as non-guaranteed backward compatible API for other languages.

TensorFlow makes it easy to create ML models that can run in any environment.

1.1 TensorFlow Lite

TensorFlow Lite is TensorFlow's lightweight solution for mobile and embedded devices. It enables low-latency inference of on-device machine learning models with a small binary size and fast performance supporting hardware acceleration.

1.1.1 TensorFlow Lite on Linux SBCs

The **NuMicro® MA35D1** series are built around dual **ARM Cortex-A35** cores (64-bit), running up to 800 MHz, these efficient cores feature the **ARM NEON SIMD** engine, allowing the **Linux SBC**s to comfortably run embedded Linux distribution and execute optimized edge AI/TensorFlow Lite tasks.

Because the Cortex-A35 prioritizes power efficiency over raw performance, standard floating-point models can choke the CPU. To achieve real-time performance on this specific hardware:

1. **Enforce Int8 Post-Training Quantization**: Integer math is drastically faster and uses less cache memory on Cortex-A series chips.
2. **XNNPACK Delegate**: Ensure the XNNPACK acceleration library is enabled in build. XNNPACK provides highly optimized, multi-threaded floating-point and fixed-point execution providers specifically designed for ARM CPUs.
3. **Multi-threading**: Scale the *num_threads* setting in TFLite Interpreter configuration to match the exact number of active CPU cores.

2 Getting Started with Buildroot Development on MA35D1

Developing a custom embedded Linux system and TensorFlow Lite TinyML applications for NuMicro® MA35D1 microprocessor is done primarily using Buildroot. Buildroot requires a 64-bit Linux host development environment, **Ubuntu 24.04 LTS** is highly recommended.

2.1 Setting Up the Build Environment

Before obtaining the source code, ensure the host system has the necessary compilation utilities, tools, and headers installed.

Run the following command on Linux host terminal:

```
sudo apt update && sudo apt install -y git build-essential libncurses-dev
```

2.2 Obtaining the Source Code

Clone the Buildroot repository for MA35D1 using the following instructions:

```
mkdir -p ~/Projects/tf_proj && cd ~/Projects/tf_proj
git clone --depth 1 https://github.com/OpenNuvoton/buildroot_2024.git buildroot
git clone --recurse-submodules https://github.com/symfund/br2-external.git
```

2.3 Patching the Buildroot

Buildroot natively supports out-of-tree builds. By default, running make without the O= parameter stores all build artifacts in the internal output/ directory and generates the .config file at the source tree root. However, during an out-of-tree build, the .config file resides directly inside the custom output directory. Under these circumstances, path references using variables like $(BASE_DIR) and $(@D) can break because Buildroot shifts its execution context. Always anchor custom paths using absolute variables like $(TOPDIR) to prevent relative path breakage during out-of-tree builds.

Apply the following patch to the Buildroot source tree to resolve this path resolution conflict:

```
cd buildroot
git am ../br2-external/patches/fix-configuration-path.patch
```

2.4 Configuring the Buildroot

Navigate to the *~/Projects/tf_proj* directory. All subsequent operations must be executed from this location.

First, list the default MA35D1 board configurations provided by Nuvoton.

```
make O=$PWD/output -C $PWD/buildroot BR2_EXTERNAL=$PWD/br2-external list-defconfigs \
   | grep ma35
```

The default MA35D1 board configurations are displayed below:

```
  numaker_hmi_ma35d16aj2_defconfig    - Build for numaker_hmi_ma35d16aj2
  numaker_hmi_ma35h04f70_defconfig    - Build for numaker_hmi_ma35h04f70
  numaker_iot_ma35d03f80_defconfig    - Build for numaker_iot_ma35d03f80
  numaker_iot_ma35d05ki1_v1_defconfig - Build for numaker_iot_ma35d05ki1_v1
  numaker_iot_ma35d14f80_defconfig    - Build for numaker_iot_ma35d14f80
  numaker_iot_ma35d14f90_defconfig    - Build for numaker_iot_ma35d14f90
  numaker_iot_ma35d16f70_defconfig    - Build for numaker_iot_ma35d16f70
  numaker_iot_ma35d16f80_defconfig    - Build for numaker_iot_ma35d16f80
  numaker_iot_ma35d16f90_defconfig    - Build for numaker_iot_ma35d16f90
  numaker_iot_ma35d16fj0_v1_defconfig - Build for numaker_iot_ma35d16fj0_v1
  numaker_som_ma35d15a910_defconfig   - Build for numaker_som_ma35d15a910
  numaker_som_ma35d15aa10_defconfig   - Build for numaker_som_ma35d15aa10
  numaker_som_ma35d15aa1a_defconfig   - Build for numaker_som_ma35d15aa1a
  numaker_som_ma35d15ab1a_defconfig   - Build for numaker_som_ma35d15ab1a
  numaker_som_ma35d16a81_defconfig    - Build for numaker_som_ma35d16a81
  numaker_som_ma35d16a910_defconfig   - Build for numaker_som_ma35d16a910
  numaker_som_ma35d16a91_defconfig    - Build for numaker_som_ma35d16a91
  numaker_som_ma35d16aa10_defconfig   - Build for numaker_som_ma35d16aa10
  numaker_som_ma35d16aa1a_defconfig   - Build for numaker_som_ma35d16aa1a
  numaker_som_ma35d16aa1_defconfig    - Build for numaker_som_ma35d16aa1
  numaker_som_ma35d16ab1a_defconfig   - Build for numaker_som_ma35d16ab1a
```

Load the default Buildroot configuration for the target board.

```
make O=$PWD/output -C $PWD/buildroot numaker_som_ma35d16aa1a_defconfig
```

Run the following command to open the Buildroot configuration menu:

```
make O=$PWD/output -C $PWD/buildroot menuconfig
```

Because typing the full path explicitly each time is cumbersome, changing the working directory to the output folder allows for the use of shorter commands:

```
cd $PWD/output
make menuconfig
```

Note that the BR2_EXTERNAL argument is absent from the command. Buildroot automatically stores and retains this path configuration after its first invocation, making it unnecessary to re-enter the argument in subsequent build operations.

2.4.1 BR2_DL_DIR

In the Buildroot configuration menu, navigate to **Build options** and locate the **Download dir** setting. Set this value to *$(TOPDIR)/../downloads*.

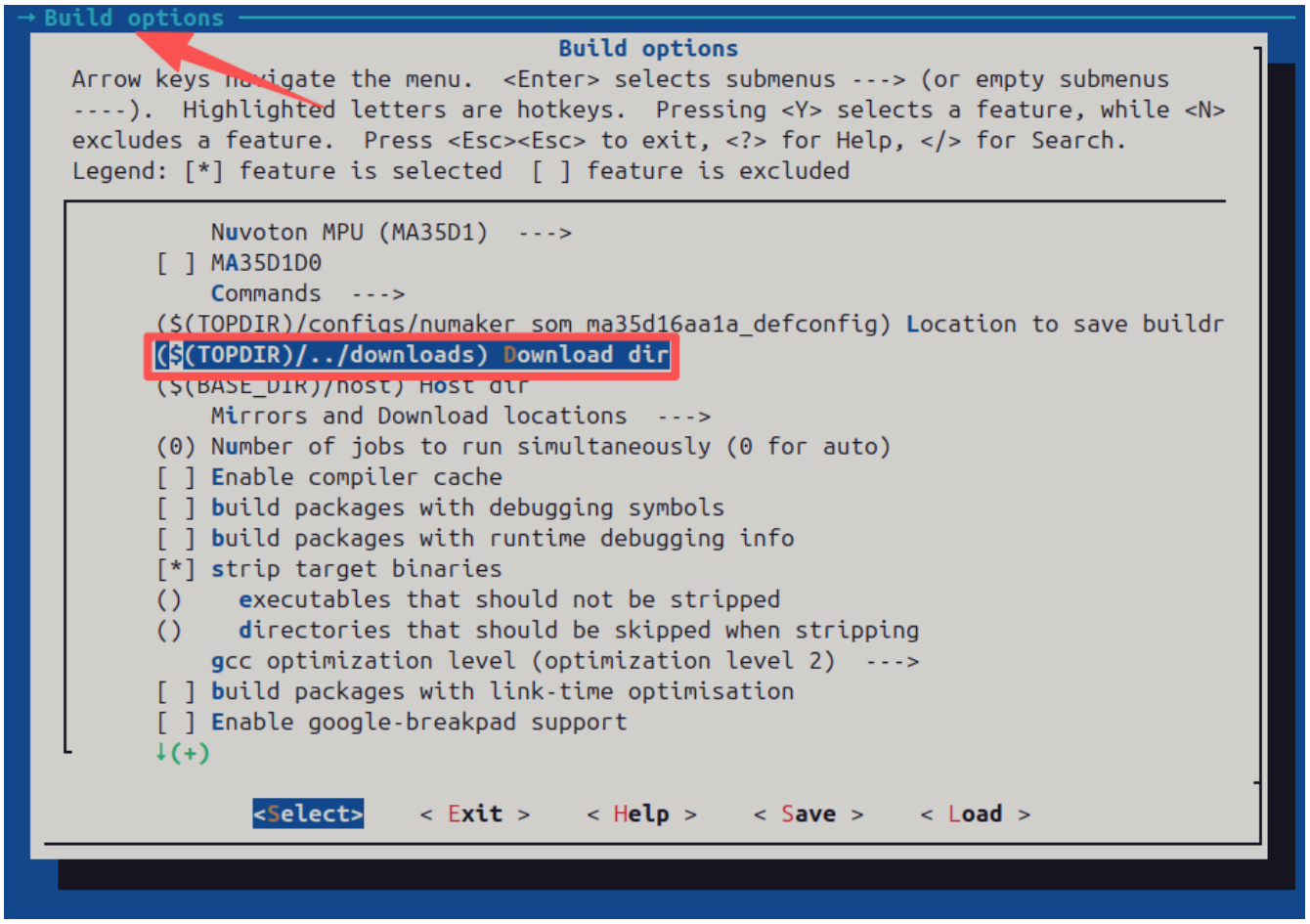

Figure 2.3.1-1 Download dir

2.4.2 BR2_PACKAGE_OVERRIDE_FILE

In the Buildroot configuration menu, navigate to **Build options** and locate the **location of a package override file** setting. Set this value to *$(TOPDIR)/../br2-external/local.mk* .

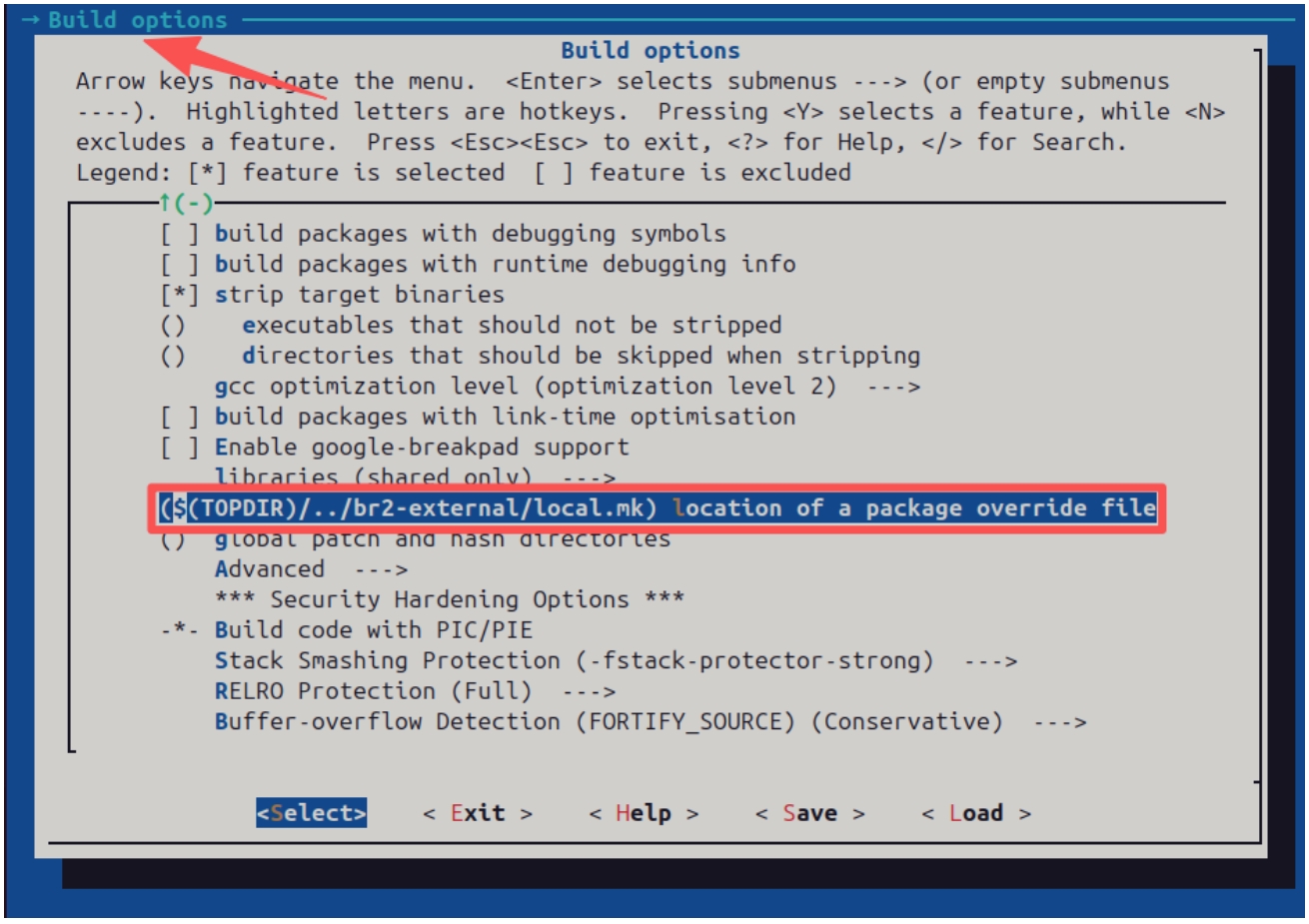

Figure 2.3.2-1 location of a package override file

2.4.3 BR2_PACKAGE_TENSORFLOW_LITE

Within the Buildroot configuration menu, navigate through **External options** to **TensorFlow Runtime Libraries** and select **tensorflow-lite**.

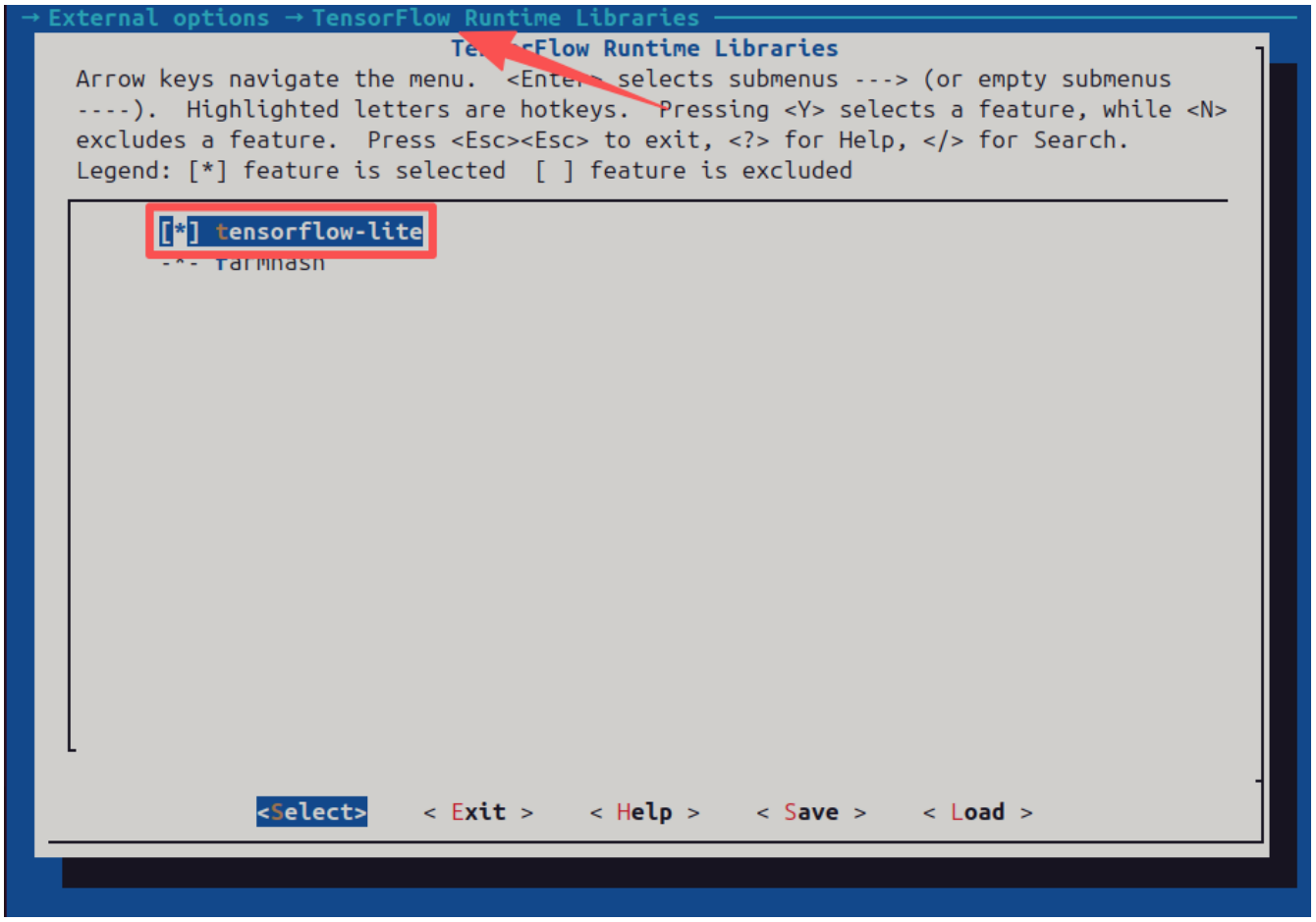

Figure 2.3.3-1 tensorflow-lite package

2.5 Building the TensorFlow Lite Runtime Library

Because the TensorFlow Lite library is integrated into Buildroot as an external package, building the TensorFlow Lite runtime library is an extremely trivial task.

Run this command to clean and build the TensorFlow Lite package individually:

```
make BR2_JLEVEL=1 tensorflow-lite-dirclean tensorflow-lite
```

- **Crucial Note**: Single-thread execution (BR2_JLEVEL=1) is required for individual builds to prevent compilation failure.
- **Optional Step**: The tensorflow-lite-dirclean target is optional but ensures a fresh build.

Accelerated Full System Build Method

To speed up the total system build time, split the process into two phases:

1. **Build System Framework First**:
   (a) Open Buildroot configuration (make menuconfig)
   (b) Unselect the tensorflow-lite package
   (c) Run a full, multi-threaded build to utilize all CPU cores:
   ```
   make all
   ```
2. **Build TensorFlow Lite Second**:
   (a) Reopen configuration and reselect the tensorflow-lite package.
   (b) Build the single package using a single thread to guarantee success:
   ```
   make BR2_JLEVEL=1 tensorflow-lite
   ```

3 Developing TensorFlow Models on Ubuntu 24.04

To begin developing TensorFlow models, the development machine should ideally be equipped with a **CUDA-capable** GPU. **Compute Capability** defines the hardware features and supported instructions for each NVIDIA GPU architecture. To Find the Compute Capability of the system GPU, visit the official NVIDIA Developer Portal.

Compute Capability	Workstation/Consumer
12.1	NVIDIA GB10 (DGX Spark)
12.0	NVIDIA RTX PRO 6000 Blackwell Workstation Edition NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition NVIDIA RTX PRO 5000 Blackwell NVIDIA RTX PRO 4500 Blackwell NVIDIA RTX PRO 4000 Blackwell NVIDIA RTX PRO 4000 Blackwell SFF Edition NVIDIA RTX PRO 2000 Blackwell GeForce RTX 5090 GeForce RTX 5080 GeForce RTX 5070 Ti GeForce RTX 5070 GeForce RTX 5060 Ti GeForce RTX 5060 GeForce RTX 5050

3.1 Installing the NVIDIA GPU Driver

A clear instruction to install the GPU driver cannot be provided without knowing the specific GPU model beforehand. Once the GPU driver installation is complete, the setup can be verified using the following command:

```
nvidia-smi
```

3.2 Installing the TensorFlow

An extensive make target, *tflite-setup-env*, is provided in the Buildroot external makefile (br2-external/external.mk) to install TensorFlow and JupyterLab.

```
make tflite-setup-env
```

Note that if the development machine is not equipped with a CUDA-capable GPU, or if the GPU driver is not installed, TensorFlow will fall back to the CPU to train models.

3.3 Launching the JupyterLab

An extensive make target, *tflite-launch-lab*, is provided in the Buildroot external makefile (*br2-external/external.mk*) to launch JupyterLab.

```
make tflite-launch-lab
```

Note that the target *tflite-launch-lab* requires a path or a specific **Jupyter Notebook** file as input. Valid inputs include $PWD/../br2-external/notebooks or $PWD/../br2-external/notebooks/03_check_cuda.ipynb.

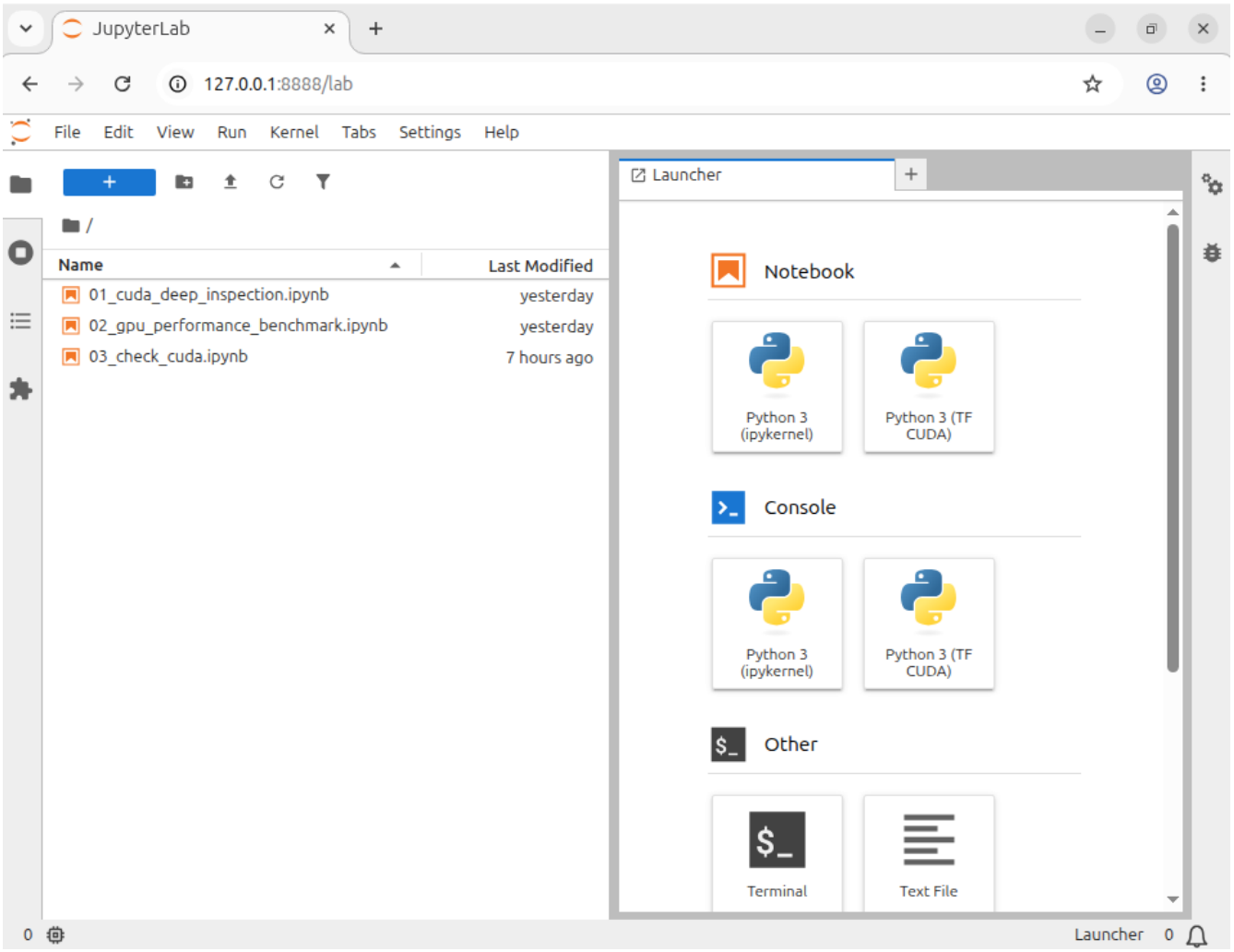

Figure 3.3-1 JupyterLab Startup

3.4 Creating the Jupyter Notebook

If TensorFlow recognizes the CUDA GPU, the kernel named TF CUDA should be selected when creating the Jupyter Notebook file.

The following Jupyter Notebook code verifies the installed version of TensorFlow, detects available GPUs, and confirm whether TensorFlow was built with CUDA support.

```
import tensorflow as tf
print("TensorFlow version:", tf.__version__)

gpus = tf.config.list_physical_devices('GPU')
print("Num GPUs Available: ", len(gpus))

if len(gpus) > 0:
   for i, gpu in enumerate(gpus):
       print(f"GPU {i}: {gpu}")
else:
   print("No GPU detected. TensorFlow will default to CPU.")

is_cuda = tf.test.is_built_with_cuda()
print("TensorFlow built with CUDA support:", is_cuda)
```

3.5 Quantizing the TensorFlow Model to INT8 TFLite

Integer quantization is an optimization strategy that converts 32-bit floating-point numbers (such as weights and activation outputs) to the nearest 8-bit fixed-point numbers.

The following code demonstrates converting a TensorFlow model into a TensorFlow Lite file, and quantizing it using full integer quantization. This strategy converts all weights and activation outputs into 8-bit integer data, whereas alternative strategies may leave a portion of data in a floating-point format.

```
def representative_data_gen():
   for input_value in tf.data.Dataset.from_tensor_slices(train_datas).batch(1).take(100):
       yield [input_value]

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_data_gen
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
# Set the input and output tensors to uint8 (APIs added in r2.3)
converter.inference_input_type = tf.uint8
converter.inference_output_type = tf.uint8
tflite_model_quant = converter.convert()

# Save the quantized model tflite_model_quant to file named model_quant_int8.tflite
tflite_model_quant_file = tflite_models_dir/model_quant_int8.tflite
tflite_model_quant_file.write_bytes(tflite_model_quant)
```

3.6 Deploying the INT8 TFLite Model to MA35D1

TensorFlow Lite inference represents the complete execution pipeline where a trained, optimized model processes input data on an edge device to generate predictions. The runtime execution follows a deterministic, sequential workflow designed to minimize memory overhead and maximize hardware utilization.

Before execution can begin, the static model file is mapped from storage into system memory to build the interpreter and allocate static runtime buffers. Once the internal tensor dimensions are initialized, the input tensors are populated with raw or preprocessed source data. Invoking the execution loop triggers the sequential processing of model layers, transforming the input data across the neural network graph to produce the final inference results within the output tensors.

The structured breakdown of this end-to-end execution sequence is illustrated in the flowchart below:

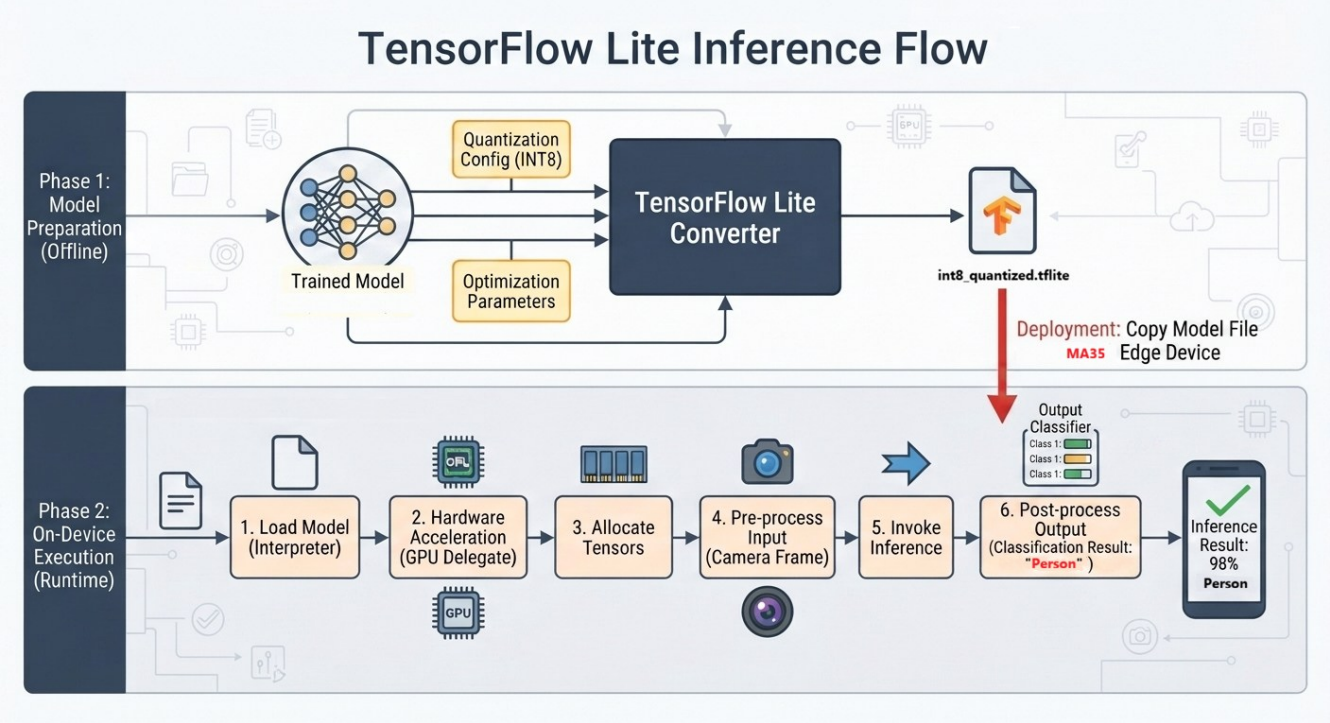

Figure 3.6-1 TensorFlow Lite Inference Flow

The following code demonstrates the comprehensive implementation details of the TensorFlow Lite inference workflow.

```
#include “tensorflow/lite/interpreter.h”
#include “tensorflow/lite/kernels/register.h”
#include “tensorflow/lite/model.h”

// 1. Load and map the model file into memory
auto model = FlatBufferModel::BuildFromFile(“/path/to/model_quant_int8.tflite”)

// 2. Pass the model pointer to the interpreter builder
tflite::ops::builtin::BuiltinOpResolver resolver;
std::unique_ptr<tflite::Interpreter> interpreter;
tflite::InterpreterBuilder builder(*model, resolver)(&interpreter);

// 3. MA35D1 has 2 CPU cores
interpreter->SetNumThreads(2);

// 4. Allocate input tensors for the model configuration
interpreter->AllocateTensors();

// 5. Populate data (zero-valued array) into the input tensors
int input_index = interpreter->inputs()[0];
TfLiteTensors* input_tensor = interpreter->tensor(input_index);

// Ensure the input tensor type matches the INT8 configuration
if (input_tensor->type == kTfLiteInt8) {
   int8_t* input_data_ptr = interpreter->typed_input_tensor<int8_t>(0);
   // Populating the input tensor with data (e.g., zero-valued array)
   int tensor_size = input_tensor->bytes;
   std::memset(input_data_ptr, 0, tensor_size);
}

// 6. Execute the model inference pipeline
interpreter->Invoke();

// 7. Access the output tensor data
int output_index = interpreter->outputs()[0];
int8_t* output_data_ptr = interpreter->typed_output_tensor<int8_t>(0);
```

4 TensorFlow Lite Model Development in Practice

This chapter demonstrates the concrete steps required to develop **TensorFlow Lite Micro (TFLM)** models for the **NuMicro® MA35D1**.

4.1 Case Study: Magic Wand

The TensorFlow Lite Magic Wand deploys a miniature 20 KB convolutional neural network (CNN) onto a microcontroller to classify physical hand gestures in real-time using multi-axis accelerometer data. It is considered the gold standard starter example for TensorFlow Lite Micro, as it perfectly balances extreme technical simplicity with a highly engaging, interactive outcome.

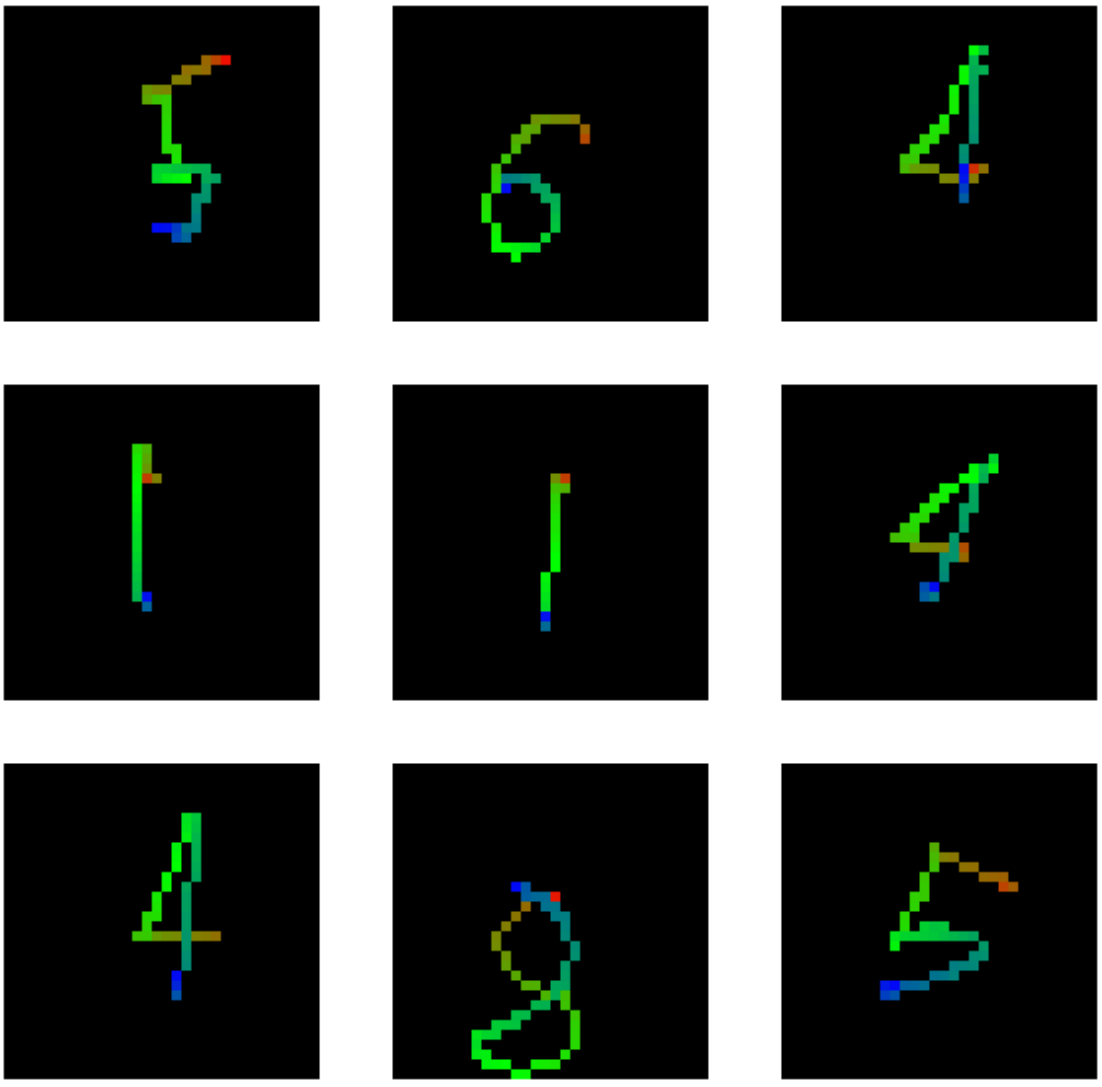

Figure 4.1-1 Hand gestures of Magic Wand

1. Ultra-Low Resource Footprint

Many beginners assume machine learning requires massive computing power. The Magic Wand completely shatters this misconception by proving complex AI can run on a chip smaller than a postage stamp.

- **Tiny Model Size:** The convolutional neural network (CNN) is highly optimized, requiring only **~20 KB of flash memory**.
- **Minimal RAM Consumption:** The entire inference process runs within a tiny **~25 KB RAM** workspace (tensor arena).
- **Hardware Friendly:** It easily fits into standard, budget-friendly 32-bit microcontrollers (like the Cortex-M55) without needing external memory chips.

2. Teaches the Core Concepts of TinyML

Instead of hiding the mechanics behind complex software layers, the Magic Wand exposes developers to the core pillars of Embedded Machine Learning:

- **Time-Series Data Processing:** It teaches how to handle a continuous stream of sensor data over time, breaking it into fixed windows (128 data points) for the model to read.
- **C++ Array Development:** Beginners learn how a trained AI model is converted from a standard file format (.tflite) into a raw hexadecimal C++ array (char model_data[]) to be compiled directly into hardware flash.
- **Low-Power Constraints:** It demonstrates how to write highly efficient code that runs quickly enough to give instant feedback without draining a small battery.

3. The Magic Wand Repository

For custom data collection and browser-based data visualization, developers can reference the open-source Magic Wand repository hosted on GitHub by Pete Warden

https://github.com/petewarden/magic_wand

4. Magic Wand Training Notebook

The Magic Wand training notebook is located at

br2-external/notebooks/04_train_magic_wand_model.ipynb

5. Training the Model via JupyterLab

1. Navigate to the build output directory *(~/Projects/tf_proj/output*) and execute the build

command make tflite-launch-lab to launch the JupyterLab environment.

2. When prompted to specify the path to the model notebook, provide the path: $PWD/../br2-external/notebooks .

3. Once the JupyterLab interface initializes in the browser, open the training notebook: 04_train_magic_wand_model.ipynb .

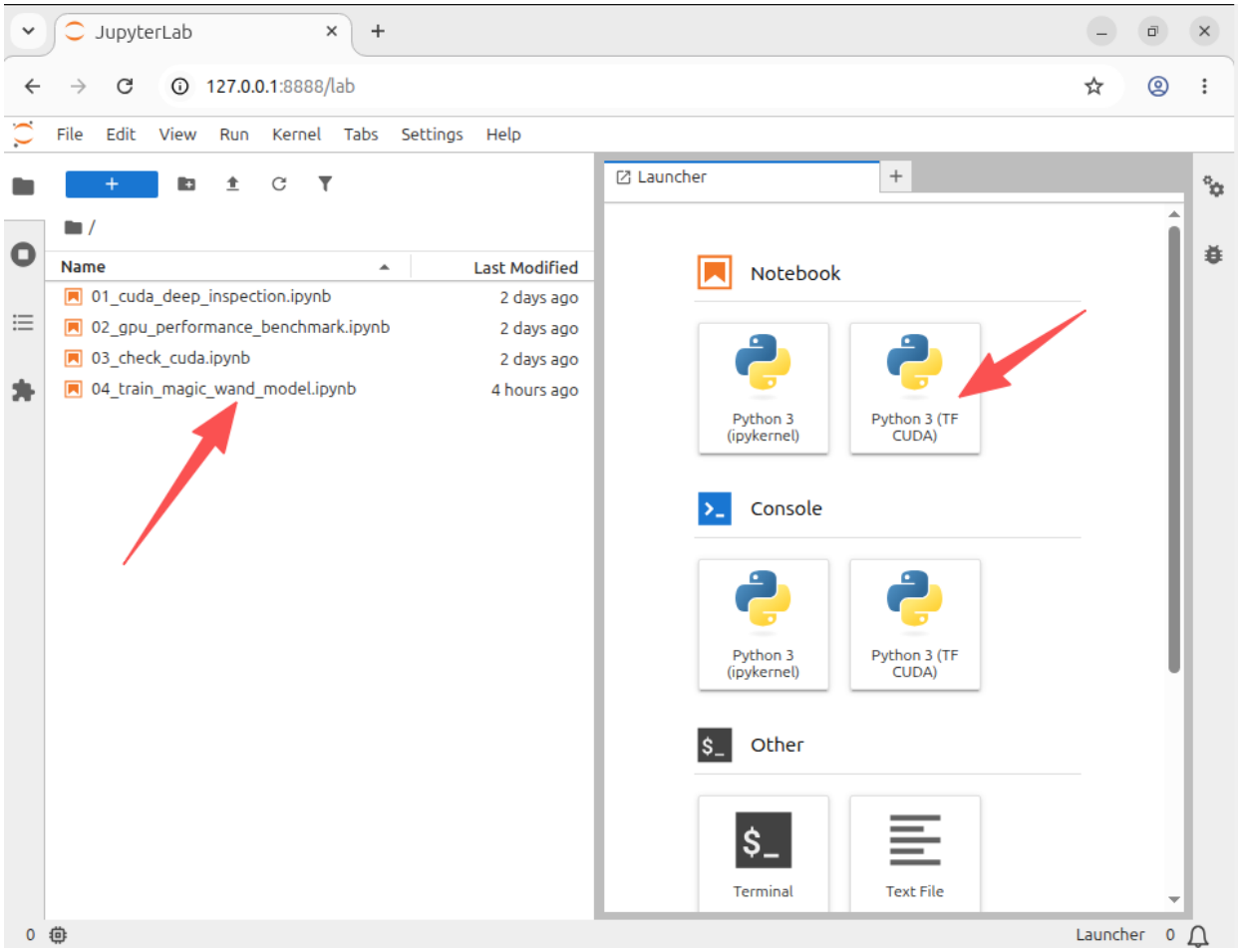

Figure 4.1-2 JupyterLab interface

4. Locate the kernel selection menu in the top-right corner of the JupyterLab interface. Switch the active kernel to **TF CUDA** to enable hardware-accelerated training via **NVIDIA CUDA** driver.

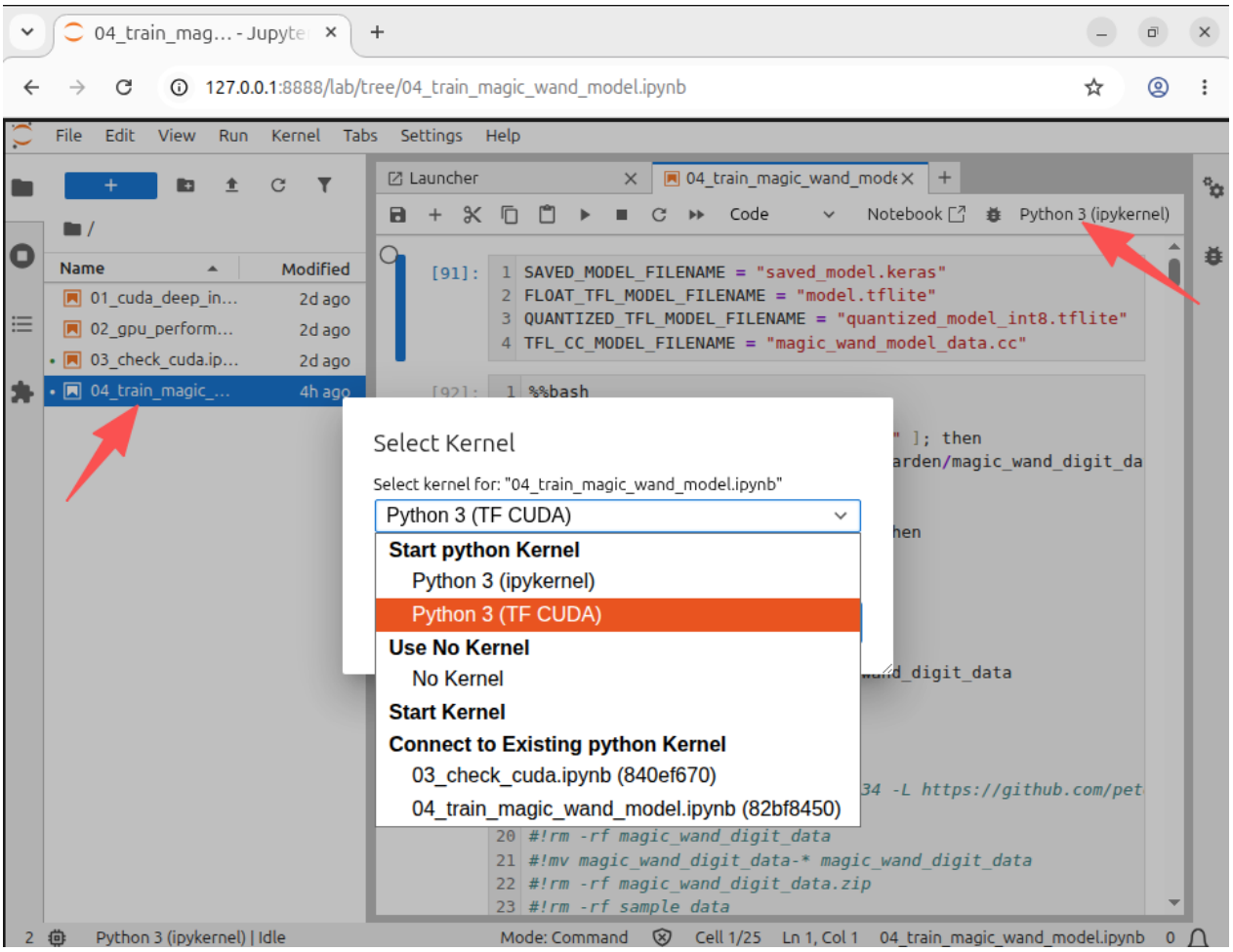

Figure 4.1-3 select CUDA kernel

5. Run the notebook cells sequentially to train the model. This is achieved by utilizing the **Shift + Enter** keyboard shortcut for each cell.

6. Upon successful completion of the training cycle, execute the quantization steps within the notebook to convert the weights to **INT8** precision. Export the finalized file as `quantized_model_int8.tflite` .

4.2 Case Study: Image Classification

Image classification models have millions of parameters. Training them from scratch requires a lot of labeled training data and a lot of computing power. Transfer learning is a technique that shortcuts much of this by taking a piece of a model that has already been trained on a related task and reusing it in a new model.

Transfer learning consists of taking features learned on one problem, and leveraging them on a new, similar problem. For instance, features from a model that has learned to identify racoons may be useful to kick-start a model meant to identify tanukis. Transfer learning is usually done for tasks where target dataset has too little data to train a full-scale model from scratch.

The most common incarnation of transfer learning in the context of deep learning is the following workflow:

1. Take layers from a previously trained model.
2. Freeze them, so as to avoid destroying any of the information they contain during future training rounds.
3. Add some new, trainable layers on top of the frozen layers. They will learn to turn the old features into predictions on a new dataset.
4. Train the new layer on target dataset.

A last, optional step, is fine-tuning, which consists of unfreezing the entire model obtained above (or part of it), and re-training it on the new data with a very low learning rate. This can potentially achieve meaningful improvements, by incrementally adapting the pretrained features to the new data.

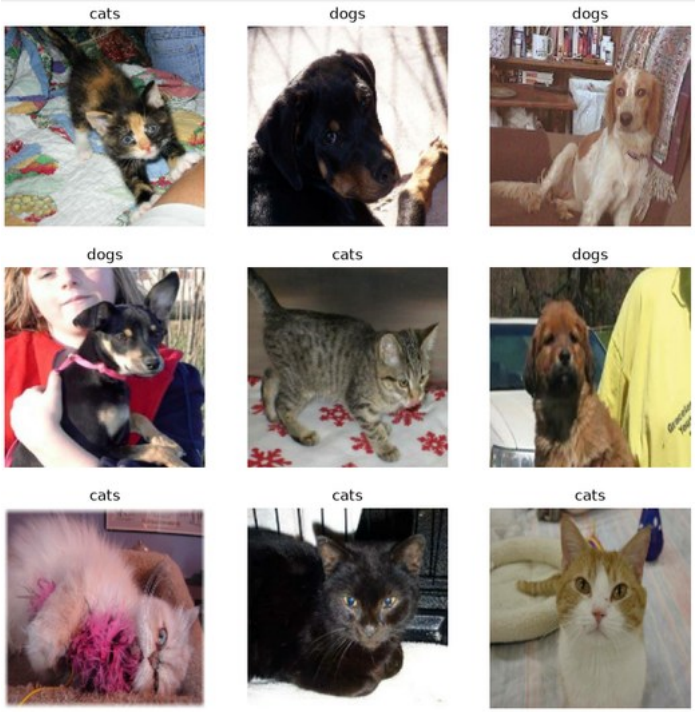

Figure 4.2-1 Cats and Dogs Classification Using Transfer Learning

- **Jupyter Notebook**

  To run the transfer learning model, open the image classification Jupyter Notebook found here:

  br2-external/models/image_classification/transfer_learning.ipynb

- **Full INT8 quantization**

  Once model training is complete, apply full INT8 quantization and save the model as

  mobilenet_v2_cats_and_dogs_full_int8.tflite

- **Evaluating the model**

  To evaluate image classification on the MA35D1 SoC target device, modify the source code of the *tflite_evaluator* (**Section 4.2.1**) project to rapidly predict cat and dog classes.

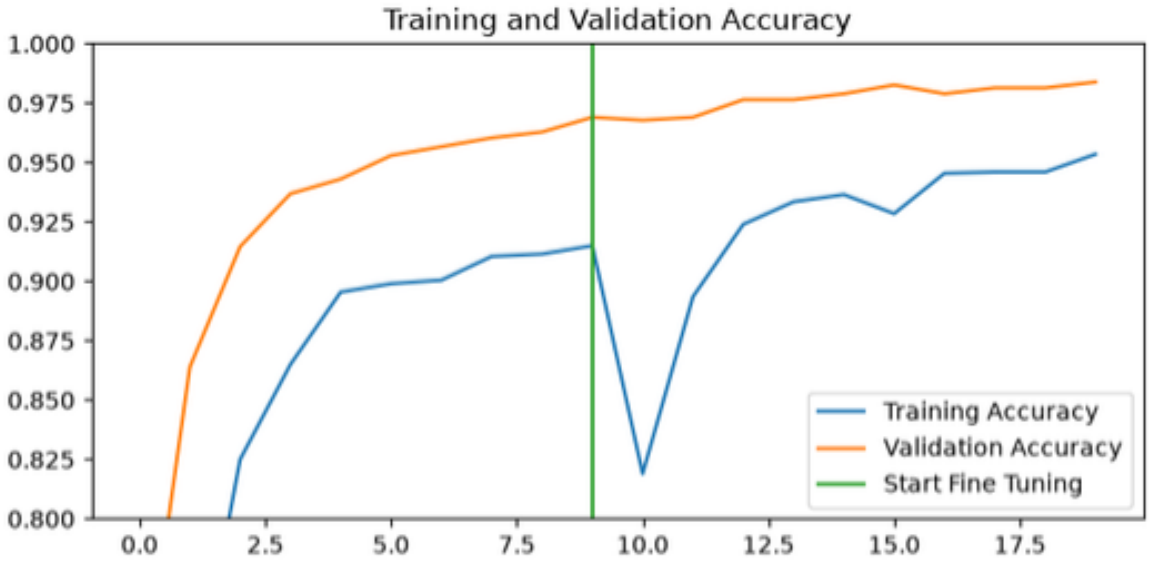

Figure 4.2-2 Inference evaluating performance for the cat and dog classification model

4.2.1 Real-World Edge AI: Deploying Int8 MobileNet v2 on Nuvoton NuMaker MA35D1

First, the TFLite FlatBuffer model is built from the source file (***.tflite**) to construct the interpreter. Next, the input tensor is retrived to extract the model's width, height, scale, and zero point. Also extract the scale and zero point from output tensor.

```
// TFLite FlatBuffer Model
std::unique_ptr<tflite::FlatBufferModel> model =
tflite::FlatBufferModel::BuildFromFile("mobilenet_v2_cats_and_dogs_full_int8.tflite");

// Interpreter
tflite::ops::builtin::BuiltinOpResolver resolver;
tflite::InterpreterBuilder builder(*model, resolver);
std::unique_ptr<tflite::Interpreter> interpreter;
builder(&interpreter);
```

```
// Input Tensor
TfLiteTensor* input_tensor = interpreter->input_tensor(0);
const int model_h = input_tensor->dims->data[1];
const int model_w = input_tensor->dims->data[2];
const float input_scale = input_tensor->params.scale;
const int32_t input_zero_point = input_tensor->params.zero_point;

// Output Tensor
TfLiteTensor* output_tensor = interpreter->output_tensor(0);
const float output_scale = output_tensor->params.scale;
const int32_t output_zero_point = output_tensor->params.zero_point;

// Feed data to input tensor
int8_t* base_input_ptr = interpreter->typed_input_tensor<int8_t>(0);
```

This implementation utilizes stb_image (**GitHub: nothings/stb**) to load image files from local storage. The system also supports real-time camera capture via OpenCV on Linux with V4L2.

```
#define STB_IMAGE_IMPLEMENTATION
#include “stb/stb_image.h”


int image_width, image_height, image_channels;
unsigned char *image_data = stbi_load(“/path/to/image.jpg”, &image_width,
                                      &image_height, &image_channels, 3);
```

Before feeding the image into the interpreter for inference, the image must be resized and normalized to match the input tensor requirements. This implementation utilizes the **bilinear resizing algorithm**.

```
for (int y = 0; y < model_h; ++y) {
    float src_y_f = ((float)y + 0.5f) * ((float)image_height / model_h) - 0.5f;
    src_y_f = std::clamp(src_y_f, 0.0f, (float)(image_height - 1));
    int y0 = static_cast<int>(src_y_f);
    int y1 = std::min(y0 + 1, image_height - 1);
    float y_weight = src_y_f - y0;

    int8_t* dest_row = &base_input_ptr[y * model_w * 3];

    for (int x = 0; x < model_w; ++x) {
        float src_x_f = ((float)x + 0.5f) * ((float)image_width / model_w) - 0.5f;
        src_x_f = std::clamp(src_x_f, 0.0f, (float)(image_width - 1));
        int x0 = static_cast<int>(src_x_f);
        int x1 = std::min(x0 + 1, image_width - 1);
        float x_weight = src_x_f - x0;

        int channel_map[3] = {0, 1, 2};

        for (int c = 0; c < 3; ++c) {
            int src_c = channel_map[c];


            float p00 = image_data[(y0 * image_width + x0) * 3 + src_c];
            float p10 = image_data[(y0 * image_width + x1) * 3 + src_c];
            float p01 = image_data[(y1 * image_width + x0) * 3 + src_c];
            float p11 = image_data[(y1 * image_width + x1) * 3 + src_c];

            float top = p00 + x_weight * (p10 - p00);
            float bottom = p01 + x_weight * (p11 - p01);
```

```
            float raw_pixel = top + y_weight * (bottom - top);
            float quantized = std::round(raw_pixel / input_scale) + input_zero_point;

            dest_row[x * 3 + c] =
static_cast<int8_t>(std::clamp(static_cast<int32_t>(quantized), -128, 127));
        }
    }
}
```

Finally, the output tensor value is dequantized using the output scale and zero point. If this dequantized value exceeds the classification threshold of 0.5, the image is classified as a dog; otherwise, it is classified as a cat.

```
// Inference
interpreter->Invoke();

// Threshold
const int8_t threshold = static_cast<int8_t>(std::round(0.5f / output_scale) +
output_zero_point);

// Dequantize
int8_t* output_data_ptr = interpreter->typed_output_tensor<int8_t>(0);
int8_t raw_int8_output = output_data_ptr[0]

// Probability
float probability = (static_cast<float>(raw_int8_output) - output_zero_point) *
output_scale;

// Confidence
float confidence = 0.0f;

if (raw_int8_output >= threshold) {
    // dog
    confidence = probability;
} else {
    // cat
    confidence = 1.0f - probability;
}
```

4.3 Evaluating the TFLite Model

Following the completion of the training and **INT8 quantization** steps for the **Magic Wand model** in **Section 4.1**, the final output file is exported as quantized_model_int8.tflite. This model can now be verified using the tflite_evaluator tool.

The tflite_evaluator utility is an out-of-tree **Buildroot package** located at br2-external/package/tflite-evaluator.

To enable and build the evaluator within the Buildroot environment, the following steps are required:

1. Launch the Buildroot configuration menu:

```
make menuconfig
```

2. Navigate to the following path to enable the utility:

   External options → TensorFlow Runtime Libraries → TensorFlow Lite tool & demos

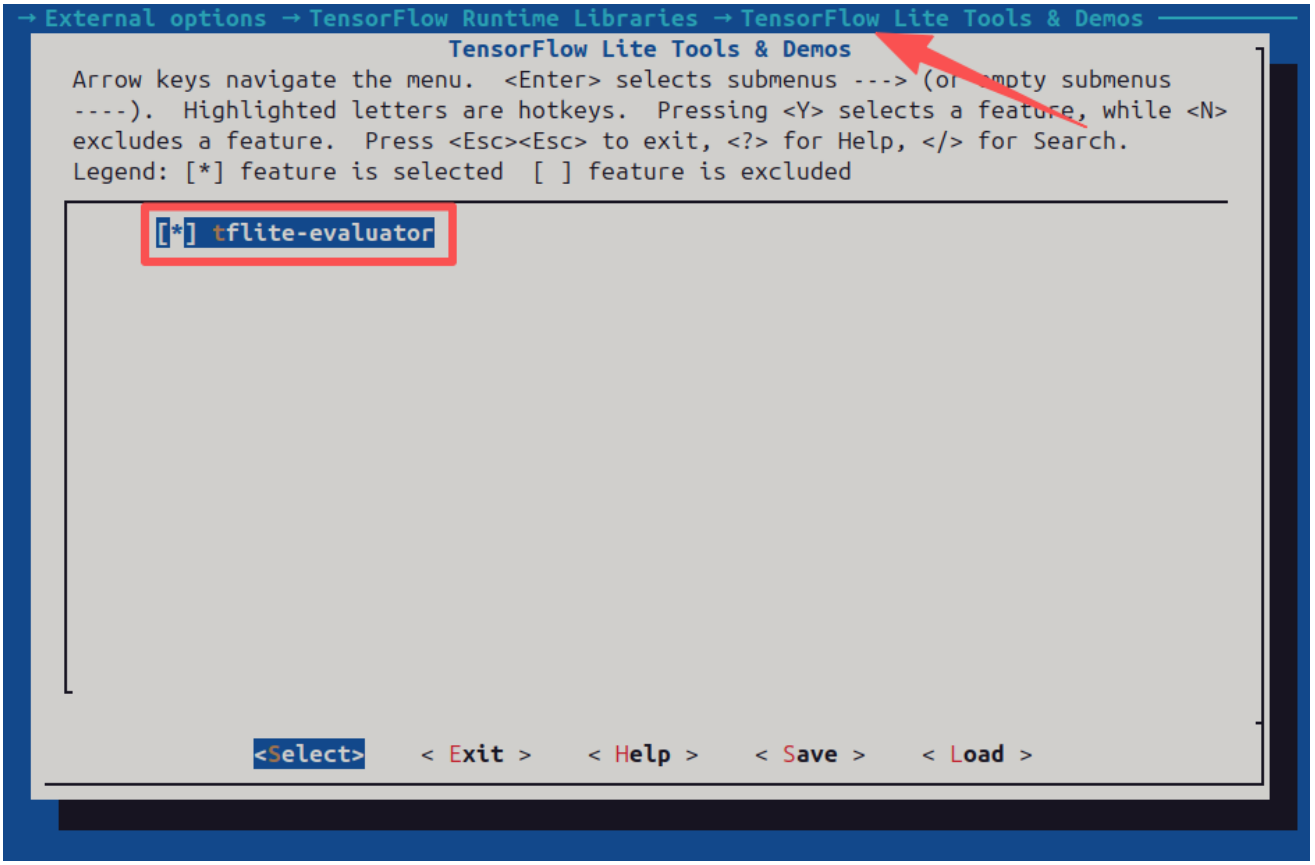

Figure 4.3-1 tflite-evaluator

3. To compile or rebuild the tflite_evaluator package inside Buildroot, execute:

```
make tflite-evaluator-dirclean tflite-evaluator && make
```

4. To evaluate the Magic Wand model on the target hardware shell, transfer the .tflite file and run the following command:

```
# tflite_evaluator /path/to/quantized_model_int8.tflite
```

5 Conclusion

Running TensorFlow Lite inference on the Nuvoton NuMicro® MA35D1 microprocessor highlights a highly effective balance between edge computational efficiency and energy conservation. By leveraging an optimized internal architecture alongside tailored hardware acceleration layers, the MA35D1 successfully minimizes resource strain while maintaining low-latency inference speeds.

When coupled with INT8 quantization, the system significantly reduces model footprint and runtime memory tracking metrics without sacrificing critical prediction accuracy. This integration demonstrates that deploying streamlined, hardware-aware machine learning frameworks onto industrial-grade edge microprocessors offers a robust, predictable pipeline well-suited for high-throughput smart gateways, edge vision processing units, and predictive maintenance installations.

Revision History

Date	Revision	Description
2026.08.20	1.06	Added the Section 4.2.1 (Deploying TFLite model)
2026.08.18	1.05	Added the Section 4.2 (Image Classification)
2026.07.24	1.04	Added the Section 4.3 (Evaluating the TFLite Model)
2026.07.22	1.03	Added the Chapter 4 (TensorFlow Lite Model Development in Practice).
2026.07.08	1.02	Updated Buildroot variable resolution guidelines in Section 2.3.
2026.07.07	1.01	Added the Section 2.3 (Patching Buildroot).
2026.07.03	1.00	Initial version.

Important Notice

Nuvoton Products are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".

Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.

All Insecure Usage shall be made at customer's risk, and in the event that third parties lay claims to Nuvoton as a result of customer's Insecure Usage, customer shall indemnify the damages and liabilities thus incurred by Nuvoton.